

HoneyPot — Operating Manual

Deception Detection Lab · running the pipeline end to end

Written for: a teammate who has the repo and wants to actually run and operate the pipeline. For first-time install, see `README/SETUP.pdf`.

Contents

1. Overview
2. Prerequisites & setup (quick)
3. Component map
4. Running the pipeline (commands)
5. Detection use cases & severity model
6. Reading the dashboards / triage
7. Testing & linting
8. Where the sensor really runs
9. Ground rules

1. Overview

This project is a student cybersecurity capstone: an **OpenCanary honeypot** impersonates three internal services that have *no legitimate users* — **SSH**, an **HTTP web-admin page**, and **MySQL**. Because nobody should ever touch these decoys, *any* interaction is inherently suspicious.

A **custom Flask frontend** we own reads OpenCanary's JSON event log directly, normalizes and sanitizes each event, runs four detection rules, and serves two dashboards: **Operations** (health + ranked alerts) and **Investigation** (per-event table with filters). The data flow is:

```
OpenCanary log → app.reader (normalize + sanitize) → app.rules (detect) → app.frontend (dashboards)
```

The sensor itself runs on an isolated Linux VM. On this dev machine you run only the frontend and the test harness against synthetic logs.

2. Prerequisites & setup (quick)

Full details are in `README/SETUP.pdf`. The short version:

- **Python 3.10+** on PATH; **Git** (+ Git Bash on Windows).
- Bootstrap: **Windows** double-click `bootstrap.bat` (or `bash setup.sh` in Git Bash); **macOS** `bootstrap.command` or `./setup.sh`; **Linux** `./setup.sh`. It creates `.venv`, installs deps, writes `requirements.lock.txt`, and copies `.env.example` → `.env`.

- Activate the venv: `source .venv/Scripts/activate` (Git Bash) / `.venv\Scripts\Activate.ps1` (PowerShell) / `source .venv/bin/activate` (macOS/Linux).
- Fill in `.env` (gitignored): replace `REPLACE_ME_*`, at minimum `OPENCANARY_LOG_PATH` and `APP_PORT`.

Known gap: `setup.sh` installs `pyyaml requests python-dotenv jsonschema` but **not** `flask`, even though the frontend needs it. If `python -m app.frontend` fails with `ModuleNotFoundError: No module named 'flask'`, run `pip install -r requirements.txt` (or `pip install flask`) inside the activated venv.

3. Component map

Path	Role	What it does
<code>app/reader/</code>	Detection lead	Reads the OpenCanary JSONL log; normalizes to the event contract and sanitizes (hashes passwords — no plaintext ever leaves here).
<code>app/rules/</code>	Detection lead	The four detection use cases as pure functions, each with a severity and alert verdict.
<code>app/frontend/</code>	Detection lead	Flask web app serving the Operations + Investigation dashboards and JSON APIs.
<code>tests/harness/</code>	Validation lead	Synthetic event generator (a file writer) — produces sample logs for local dev/testing. Never a live network client against out-of-scope hosts.
<code>sensor/</code>	Sensor lead	OpenCanary config template (<code>opencanary.conf.template</code>) + fixtures. MVP modules: ssh, http, mysql.
<code>infra/</code>	Infra lead	Terraform / cloud-init provisioning stubs for the Linux VM.
<code>docs/</code>	—	Architecture, network policy, runbook, event contract, cost model.
<code>evidence/</code>	—	Sanitized acceptance-test screenshots/logs only.

4. Running the pipeline (commands)

Run all commands from the repo root with the `.venv` activated.

Step 1 — Generate a synthetic log

No real honeypot is needed for local dev; the harness writes a JSONL log file:

```
python -m tests.harness --count 50 --decoys ssh,http,mysql --seed 1 --out sample.log
```

Flag	Meaning
<code>--count</code>	Number of events to write (default 50).
<code>--decoys</code>	Comma-separated subset of <code>ssh,http,mysql</code> (default all three).
<code>--out</code>	Required. Output JSONL log path.
<code>--seed</code>	RNG seed for reproducible output (optional).
<code>--node-id</code>	Non-identifying sensor label (default <code>decoy-01</code>).

Step 2 — Inspect normalized events (optional)

Normalize a log and print one JSON object per line to stdout (malformed lines are summarized on stderr). This is the stable interface the GUI builds against:

```
python -m app.reader sample.log
```

Step 3 — Launch the dashboards

```
python -m app.frontend --log sample.log
```

Flag	Meaning
<code>--log</code>	Required. Path to an OpenCanary JSONL log.
<code>--host</code>	Bind host (default <code>127.0.0.1</code> — keep it localhost; do not expose publicly).
<code>--port</code>	Bind port (default <code>8000</code>).
<code>--debug</code>	Flask debug mode.

Then open in a browser:

- `http://127.0.0.1:8000/` — **Operations** (health tiles + ranked findings)
- `http://127.0.0.1:8000/investigation` — **Investigation** (per-event table with filters)

JSON endpoints power the ~5s auto-refresh and any external consumer: `/api/summary` and `/api/events` . The log is re-read on each request, so regenerating `sample.log` updates the dashboards live.

5. Detection use cases & severity model

The decoys have no legitimate users, so interaction is suspicious by default and severity escalates from there (credential use > passive touch; coordinated multi-service recon is highest).

Use case	Severity	Fires when
<code>unexpected_interaction</code>	MEDIUM	Any interaction with a decoy service (e.g. a bare connection).
<code>authentication_attempt</code>	HIGH	Credentials were sent (<code>event_category = auth_attempt</code>) — e.g. a login attempt on the SSH decoy.
<code>multi_service_probing</code>	HIGH	One <code>src_ip</code> touches ≥ 2 distinct decoy services (coordinated recon).
<code>pipeline_health</code>	INFO	Unmapped / startup events (e.g. OpenCanary boot messages). Observability, not a threat.

6. Reading the dashboards / triage

Condensed from `docs/runbook.md`. Core premise: triage is about scope and intent, not "is this real."

- **Start on Operations (/).** Check `Alerts`, `Distinct sources`, and `Last event` freshness. A stale last-event may mean the sensor or log path is down — not that things are calm.
- **Rule out test traffic.** The harness is a file writer and should never appear as a live source. Known benign lab/test hosts may be excluded — record the exclusion; never hard-code it into detection.
- **Drill into Investigation (/investigation).** Filter by the `src_ip` of interest, then check: `event_category` (`auth_attempt` = credentials sent, highest intent); `service` spread (same IP across services $\Rightarrow$ multi-service probing); identical `password_sha256` across events (credential reuse, correlated without plaintext); `timestamp` clustering (rapid bursts $\Rightarrow$ automation).

Escalation:

- **HIGH** $\rightarrow$ escalate immediately. Any `auth_attempt` or `multi_service_probing` is a strong indicator of active recon/attack; notify security on-call and preserve `evidence/` for the source IP.
- **MEDIUM** $\rightarrow$ review within the shift; watch for the same `src_ip` escalating.
- **INFO** $\rightarrow$ no action. A rising `unmapped_rate` or missing recent events is a pipeline-health issue — check the sensor and log transport.

Passwords are only ever stored and displayed as hashes. Never expect, request, or introduce plaintext credentials.

7. Testing & linting

```
pytest          # run the test suite
ruff check .    # lint
ruff format .   # format
```

Tests use sanitized fixtures under `tests/fixtures/`; dev deps (`pytest`, `ruff`, `pre-commit`) are in `requirements-dev.txt`.

8. Where the sensor really runs

The OpenCanary sensor is intended to run on an **isolated Linux VM with no public exposure** — that VM is the authoritative install/deploy target. This dev machine (Windows/Mac) is only for writing code and running the frontend + harness locally. OpenCanary has known install friction on macOS/Windows; `setup.sh` installs it best-effort and a local failure there is **expected and non-fatal**.

9. Ground rules

- **Everything is synthetic.** No real credentials, no CUI, no production data.
- **Sanitization is the point.** Never introduce raw/unsanitized capture data or secrets into tracked files.
- **Secrets stay local.** `.env` and the filled-in `opencanary.conf` are gitignored; only `.env.example` and the `.template` are tracked.
- **The harness targets approved lab hosts only.** Never point any traffic-generating client at hosts outside the approved lab scope.
- **Default to clearly-labeled placeholders** (`REPLACE_ME_*`, `PLACEHOLDER`) rather than inventing real ports, IPs, hostnames, or tokens.